

Input and Output Devices:

Input and Interaction: Input Devices. Physical Input Devices, Logical Devices. Measure and trigger. Input Modes. Event-Driven Input: Using the pointing device, Window events, and Keyboard events. Menus.

VR related Input Devices: Trackers, Navigation, and Gesture Interfaces; **VR related Output Devices:** sound displays & haptic feedback

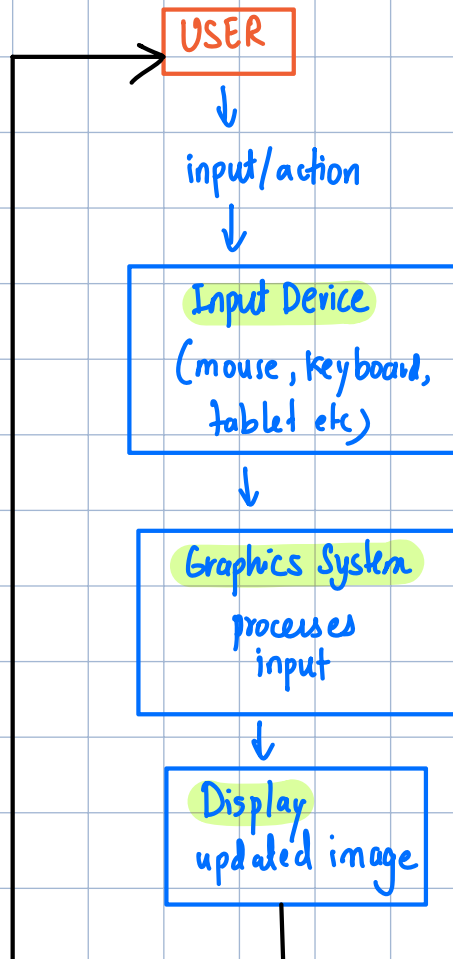
INPUT INTERACTION :

A computer graphics system doesn't just display an image. In an interactive graphics system, the user can change what is displayed

eg: • you move mouse → cursor moved

• you resize the window → graphics are redrawn

→ Basic Interaction Loop:



INPUT DEVICES:

An input device is a hardware device used by the user to provide information or commands to the graphics system.

common graphics input devices include:

- Keyboard
- mouse
- trackball
- joystick
- graphics tablet
- Stylus
- touch screen
- data glove
- 3D mouse
- Digitizer
- voice input



* Physical Input devices and Logical input devices are not the same thing.

⇒ PHYSICAL INPUT DEVICES:

A physical input device is the actual hardware that the user touches or operates

eg: Keyboard, mouse, joystick, trackball, graphics tablet, touch screen, data glove, space ball, dial, button box

Physical Input Devices

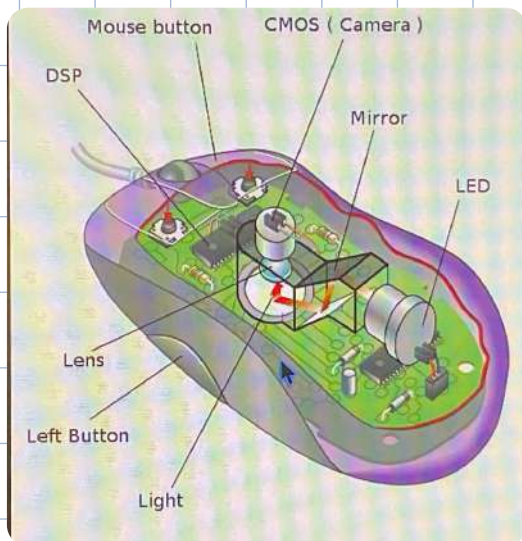
Allow users to indicate a position or direction

Pointing Devices

- ↳ mouse
- ↳ trackball
- ↳ joystick
- ↳ light pen
- ↳ graphics tablet
- ↳ stylus
- ↳ touch screen

Keyboard Devices → these provide character or Key-based input

- ↳ Keyboard
- ↳ Numeric Keypad
- ↳ Function Keys
- ↳ special purpose key panels



⇒ LOGICAL DEVICES:

A logical input device describes the input in terms of what the application program needs, rather than the physical characteristics of the hardware - this abstraction allows different physical devices to provide the same logical type of input

→ also called LOCATOR

Physical Device



Mouse



provides raw information



LOGICAL DEVICE



Locator



provides position to program

→ Traditional graphics system such as GKS/PHIGS defines 6 major logical input device classes:

① Locator:

a locator provides a position to the application
↳ (x, y) or (x, y, z)

eg: mouse, joystick, graphics tablet, touch screen, light pen.

Applications:

- drawing a point
- positioning an object
- moving an object
- placing text

② Pick:

a pick device identifies a particular graphical object

Applications :

- object selection
- editing
- moving objs
- deleting objs
- changing obj properties

③ Choice:

A choice device allows the user to select one option from a set of options

eg: 'menus

- buttons
- function keys
- touch screen options

④ Valuator:

A valuator provides a numerical value, generally a scalar

eg: slider, dial, scrollbar, analog controller, mouse wheel

Applications :

- zoom
- rotation angle
- scale factor
- brightness

⑤ String:

a string device provides a sequence of characters

Application:

• entering names

• commands

• labels

• filenames

• text.

⑥ Stroke:

a stroke device provides an array/sequence of positions

Applications:

• freehand drawing

• handwriting

• sketching

• gesture recognition

24. All Six Logical Devices — Memorize This Table

Logical device	Returns	Example	Easy memory
Locator	Position	Mouse	Where?
Pick	Object ID	Mouse click	Which object?
Choice	One option	Menu	Which option?
Valuator	Numeric value	Slider/dial	How much?
String	Characters	Keyboard	What text?
Stroke	Sequence of positions	Tablet	What path?

This table is extremely exam-worthy.

25. Physical vs Logical — Important Comparison

Physical device	Logical device
Actual hardware	Software/application-level abstraction
Describes hardware	Describes type of information
Mouse	Locator
Keyboard	String
Tablet	Stroke/Locator
Dial	Valuator
Menu	Choice
Mouse selecting object	Pick

MEASURE & TRIGGER:

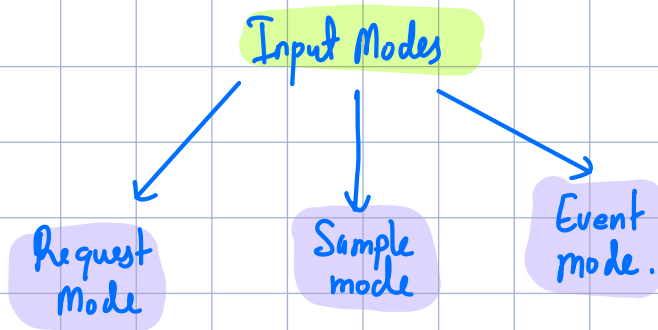
→ Measure: measure is the information returned by the input device to the application

eg. for keyboard → character/string
for mouse → position

→ Trigger: It is the physical action used to signal the computer.
eg. mouse button click → trigger

INPUT MODES:

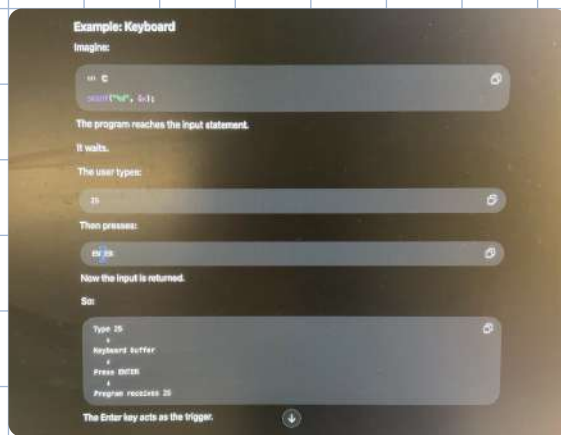
Input mode describes how and when the measure is delivered to the application



① Request mode:

→ in request mode, the application explicitly asks for input

→ the program waits until the user provides the input/trigger

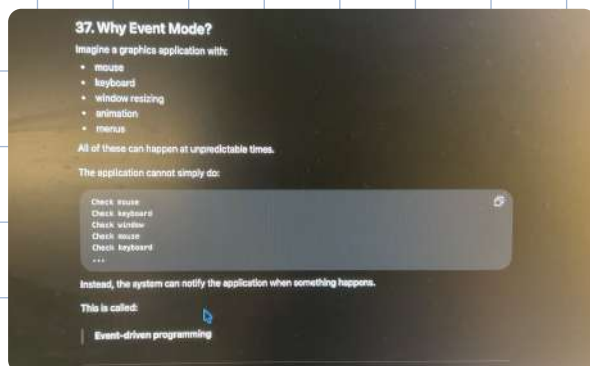


② Sample Mode:

- In sample mode, the application obtains the current value immediately when it asks for it.
- there is no separate required.

③ Event Mode:

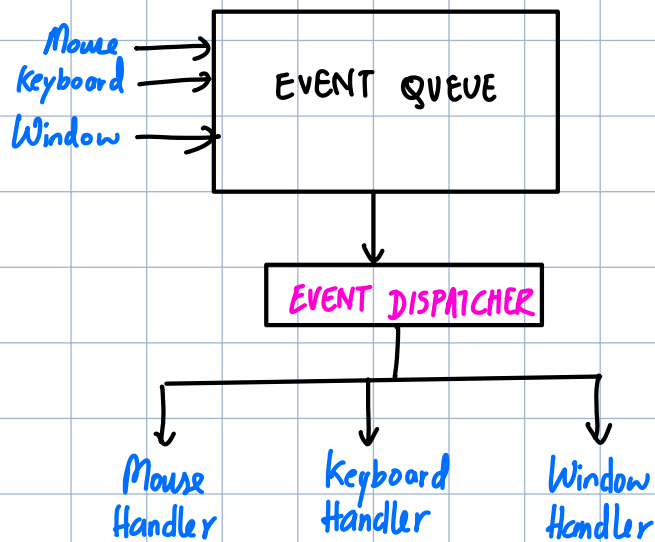
- in the event mode, the program does not continuously ask: "Did the user click?"
- ↳ instead, the graphics/window system detects events and puts them in an **Event Queue**



Events in OpenGL

Event	Example	OpenGL Callback Function
Keypress	KeyDown KeyUp	glutKeyboardFunc
Mouse	leftButtonDown leftButtonUp	glutMouseFunc
Motion	With mouse press Without	glutMotionFunc glutPassiveMotionFunc
Window	Moving Resizing	glutReshapeFunc
System	Idle Timer	glutIdleFunc glutTimerFunc
Software	What to draw	glutDisplayFunc

EVENT DRIVEN PROGRAMMING :



→ the program responds to events when they happen

⇒ CALLBACK FUNCTIONS:

→ a callback is a function that the programmer provides to be executed when a particular event occurs

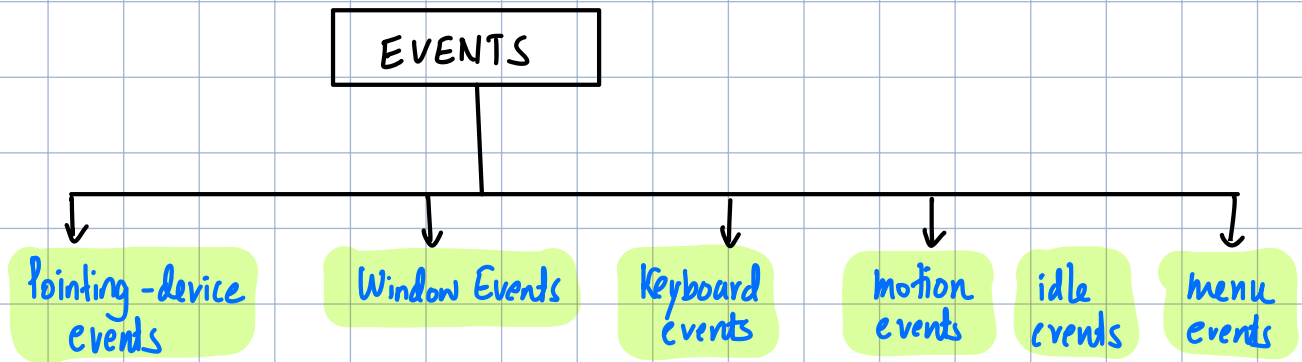
eg: mouse event

↓
mouse callback() $\xrightarrow{\text{GLUT}}$ glutMouseFunc()

* GLUT Event Model :

```
int main() {  
    initialize();  
    glutDisplayFunc(display);  
    glutKeyboardFunc(keyboard);  
    glutMouseFunc(mouse);  
    glutReshapeFunc(reshape);  
    glutMainLoop();  
}
```

⇒ EVENT TYPES:



⇒ POINTING DEVICE EVENTS:

→ pointing devices include :

- mouse
- trackball
- tablet
- touch input
- joystick
- stylus

→ Typical mouse input include:

① Mouse Btn Event:

Suppose:

LEFT CLICK at (250,300)

The event contains information such as:

Button = LEFT
State = DOWN
Position = (250,300)

A graphics program can use this to:

- select an object
- create a point
- start dragging
- activate a menu

② Mouse Motion Event:

46. Mouse Motion Event

Suppose the mouse moves:

(100,100)
↓
(110,105)
↓
(120,110)
↓
(130,120)

The graphics system can receive these changing coordinates.

Applications include:

- dragging
- drawing
- rotating a model
- camera movement
- cursor interaction

③ Dragging:

47. Dragging

Dragging is particularly important.

Sequence:

Mouse button DOWN
↓
Start dragging
↓
Mouse moves
↓
Motion events
↓
Mouse button UP
↓
Stop dragging

For example:

•
|
• drag
|
•

The object follows the mouse.

④ Passive motion

48. GLUT Mouse Callback

A traditional GLUT callback looks like:

```
void mouse(int button, int state, int x, int y)
{
    if(button == GLUT_LEFT_BUTTON &&
        state == GLUT_DOWN)
    {
        // handle click
    }
}
```

And it is registered using:

```
glutMouseFunc(mouse);
```

Conceptually:

Mouse event

↓

GLUT

↓

mouse(...)

↓

Your code

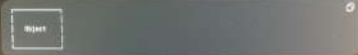
You don't need to memorize every API function unless your teacher expects programming, but understand the event → callback relationship.

WINDOW EVENTS:

- a graphics application exists inside a window
- the window can generate events
- important window events include:
 - window creation
 - window resize
 - window exposure / redrawing
 - minimize / iconify
 - moving / resizing
 - close

50. Window Resize Event

Suppose the original window is:



User resizes it:



The graphics program needs to respond.

Why?

Because the viewport and projection may need adjustment.

Conceptually:



51. Display/Redraw Event

A graphics window may need to be redrawn when:

- it becomes visible
- another window moves away
- the window is resized
- the application requests redraw

The system invokes the display callback.

For example:

```
// C
glutDisplayFunc(display);
```

Then:

```
Display event
+
display()
+
draw scene
```

KEYBOARD EVENTS:

→ Keyboard events occur when keys are pressed or released

53. ASCII Keyboard Events

For ordinary keys:

```
// C
void keyboard(unsigned char key, int x, int y)
{
    if(key == 'q')
        exit(0);
}
```

Registration:

```
// C
glutKeyboardFunc(keyboard);
```

The event information includes:

```
key
x
y
```

where:

- **key** = character pressed
- **x,y** = cursor position

55. Special Keys

Not every key is an ordinary ASCII character.

Examples:

- Arrow keys
- Function keys
- Home
- End
- Page Up
- Page Down

These are handled separately in GLUT-style systems using special-key callbacks.

Conceptually:

```
Normal keys
+
keyboard callback

Special keys
+
special-key callback
```

57. Polling vs Event-Driven Input

This is a useful related concept.

Polling

The program repeatedly asks:

```
Did the mouse move?  
Did the keyboard change?  
Did the window resize?  
Did the mouse move?  
Did the keyboard change?  
...
```

Like:

```
while(1)  
{  
    checkMouse();  
    checkKeyboard();  
    checkWindow();  
}
```

Event-driven

The program waits for events:

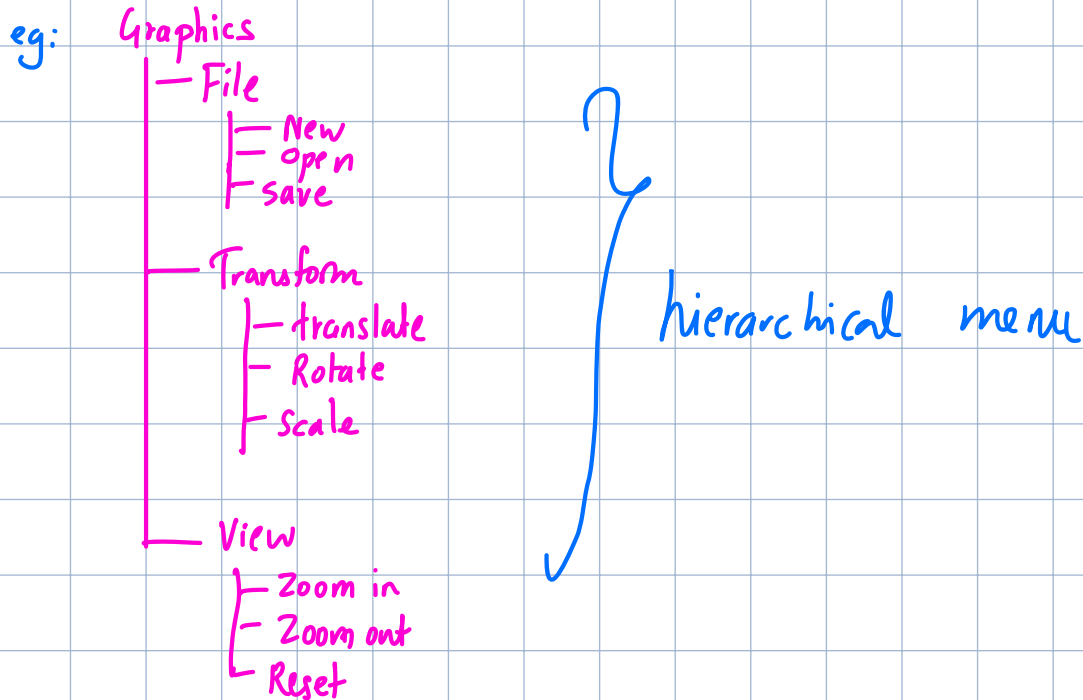
```
Event occurs  
  ↓  
System detects it  
  ↓  
Event queued  
  ↓  
Callback executed
```

MENUS :

- A menu is a graphical interface mechanism that allows the user to choose one option from a list.
- menus are essentially a practical implementation of the choice logical input device
- menus are used because users don't need to memorize commands

⇒ Menu Hierarchy:

→ Menus can have submenus



- Submenu is a menu associated with another menu option

⇒ Menu Interaction:

User clicks menu



Menu opens



User moves pointer



Option highlighted



User selects option



Menu event generated → Menu callback

→ Application performs operations

65. Menus in GLUT

GLUT provides support for popup menus.

The basic conceptual process is:

```
Create menu
├
├ Add menu entries
├
├ Register menu callback
├
├ Attach menu to mouse button
├
├ User opens menu
├
├ User selects item
├
└ Callback executes
```

GLUT provides popup-menu support as part of its device/window handling facilities.

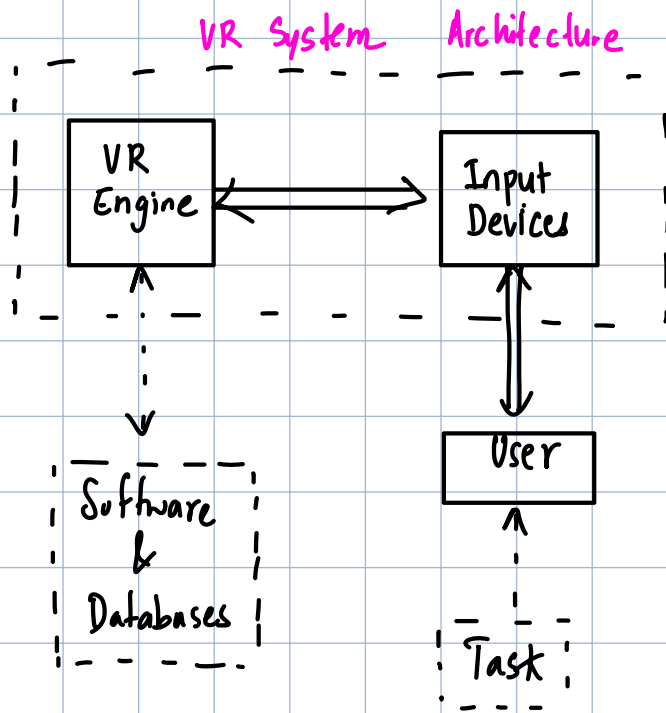
A simplified example:

```
on C
void menu(int option)
{
    switch(option)
    {
        case 1:
            // draw circle
            break;

        case 2:
            // draw rectangle
            break;

        case 3:
            exit(0);
    }
}
```

VR INPUT DEVICES: TRACKERS, NAVIGATION & GESTURE INTERFACES:



→ in order to allow human-computer interaction it is necessary to use special interfaces designed to input a user's commands into the computer and to provide feedback from the simulation to the user.

⇒ 3D Positional Trackers:

→ many computer application domains, such as navigation, ballistic missile tracking, ubiquitous computing, robotics, biomechanics, architecture, computer-aided design (CAD), education, and VR, require knowledge of the real-time position and orientation of moving objects within some frame of reference.

→ requirement parameters:

- measurement range
- precision
- temporal update rates

→ A moving object in 3D space has 6 degrees of freedom

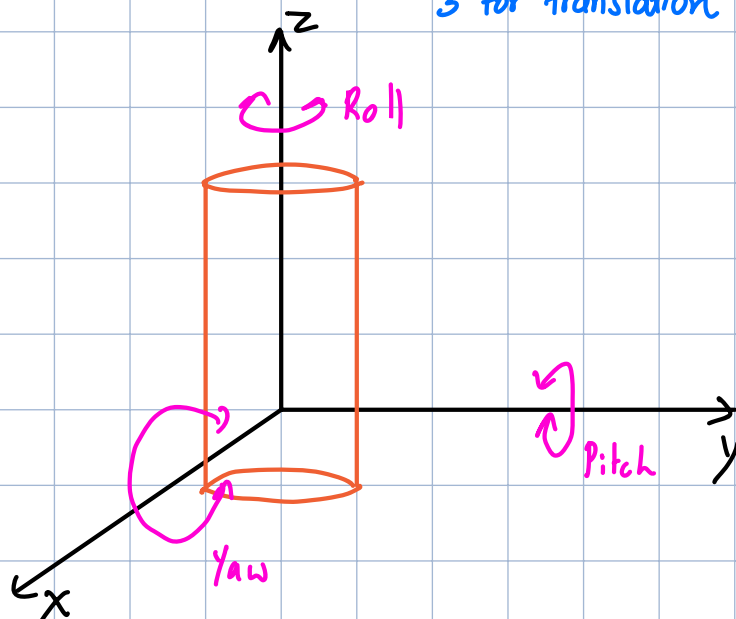
↓ ↓
3 for translation 3 for rotation

• rotation about:

X-axis → yaw

Y-axis → pitch

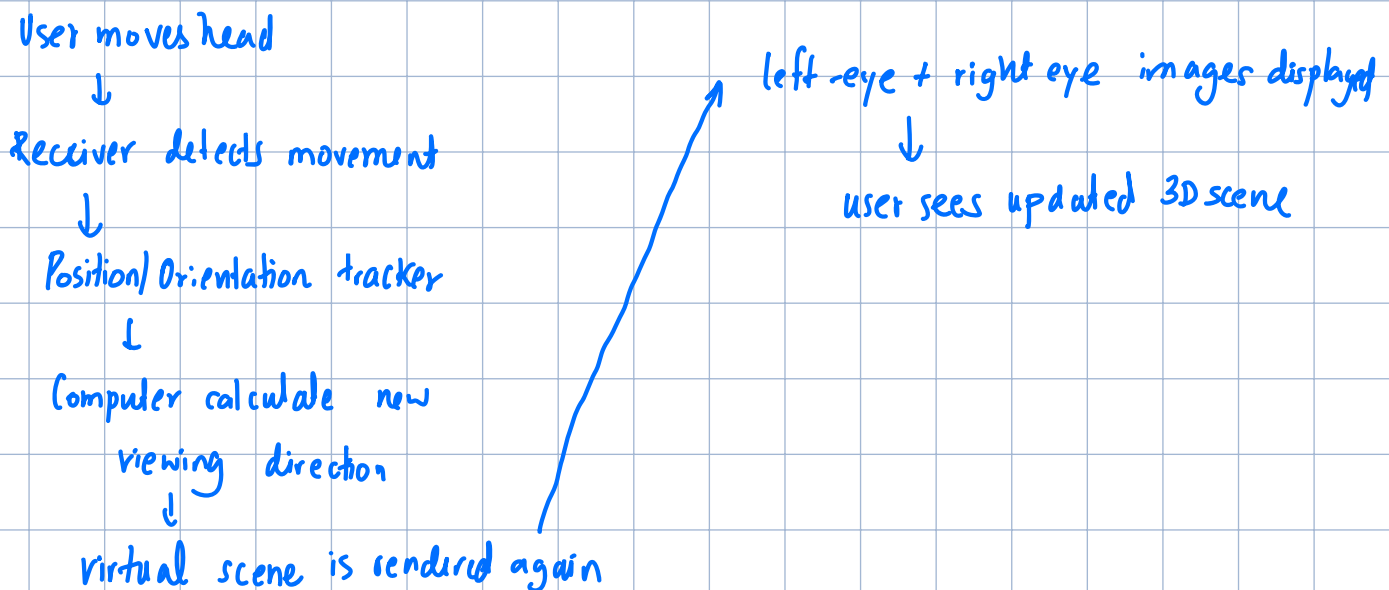
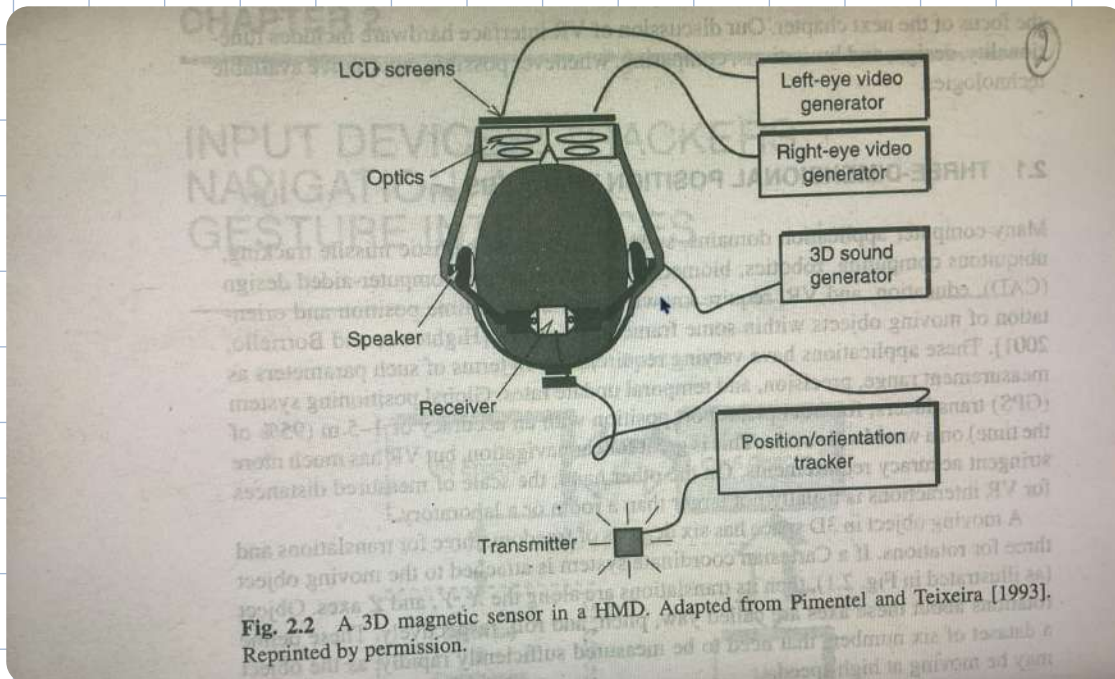
Z-axis → roll



[Definition]: The special-purpose hardware used in VR to measure the real-time change in a 3D object position and orientation is called a tracker.

→ Applications of trackers:

- (1) motion of the user's head, hands or limbs (view control, locomotion, obj manipulation)
- (2) control of an avatar, or virtual body.



→ 3D sound:

- the tracker info can also be used for 3D audio
- the computer can determine where virtual sound sources should appear relative to the user.
- this makes the virtual environment feel more realistic and immersive.

→ Tracker Performance Parameters:

All 3D trackers, regardless of the technology they use, share a number of very important performance parameters, such as accuracy, jitter, drift, and latency. These are illustrated in Figure 2.3.

Definition Tracker accuracy represents the difference between the object's actual 3D position and that reported by tracker measurements.

The more accurate a tracker, the smaller this difference is and the better the simulation follows the real user's actions. Accuracy is given separately for tracking translation (fraction of a millimeter) and rotation (fraction of a degree). Accuracy is typically not constant and is degraded with distance from the origin of the reference

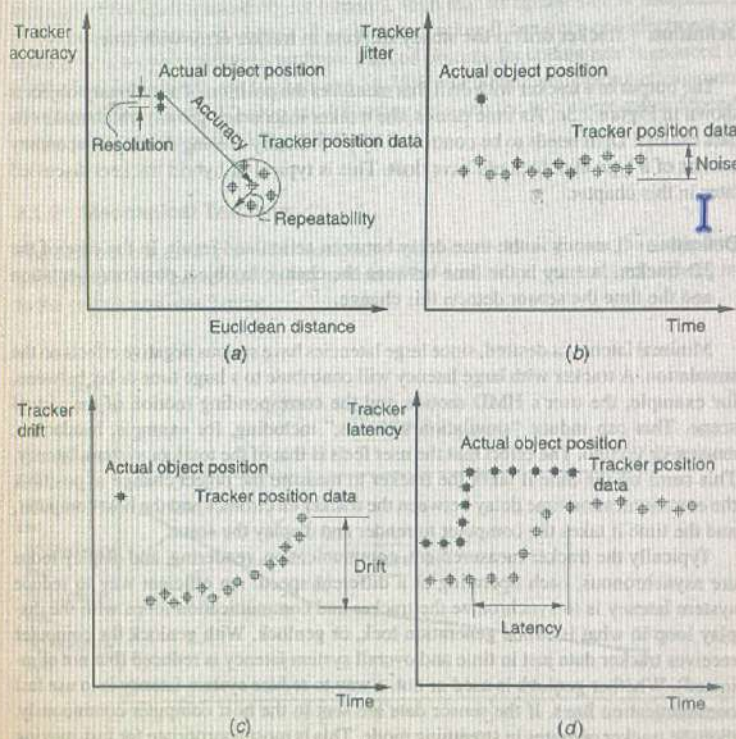


Fig. 2.3 Tracker performance parameters: (a) accuracy; (b) jitter; (c) drift; (d) latency.

⇒ Navigation and Manipulation Interfaces:

[Definition]: A navigation/manipulation interface is a device that allows the interactive change of the view to the virtual environment and exploration through selection and manipulation of a virtual object of interest.

→ can be done in either absolute coordinates or relative coordinates

→ navigation/manipulation in relative coordinates allows for incremental position control relative to the object's previous 3D position.

2.2.1 Tracker-Based Navigation/Manipulation Interfaces

Trackers offer more functionality to VR simulations than simply measuring the real-time position/orientation of the user's hand and head. Integrated within a structure that houses user-programmable pushbuttons, trackers become navigation and manipulation interfaces. Examples are the Polhemus 3Ball and the Ascension Technology 3D Mouse [Anon, 1998]. The 3Ball is a hollow billiard ball that houses a tracker receiver inside and has a pushbutton on the surface. An example of its use is to move a virtual camera that travels along the 3D vector controlled by the ball as long as the pushbutton is pressed. Another example is a wand, which projects a virtual ray along the tracker-controlled vector. Any object intersected by this ray can be selected by pushing the button on the billiard ball, and then repositioned through wrist motion. The 3D Mouse offers more functionality, as it can work both as a mouse (in 2D) and as a navigation/manipulation interface in 3D, similar to the 3Ball. Further functionality

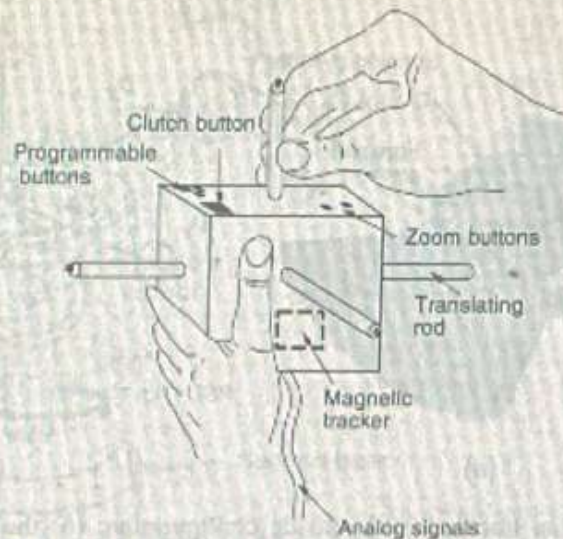


Fig. 2.21 The Cubic-Mouse. Adapted from Fröhlich et al. [2000]. © 2000 IEEE. Reprinted by permission.

is obtained through the programming of the three buttons incorporated on the 3D Mouse. (5)

One drawback of absolute position control of virtual objects is the user's limited arm reach. The larger the dimensions of the graphics display, the more this becomes a problem. One way to solve the disparity between the user's arm reach and the (larger) display dimensions is to multiply the tracker readings with a constant (gain) larger than unity. The drawback is that tracker noise is equally amplified, and selected objects appear more jittery than they would otherwise be. An alternative is to use indexed motions, in which virtual objects are selected and deselected repeatedly. When the object of interest is deselected, it stays in place, while the user moves his or her arm back and then reselects the object and moves it using the tracker, and so on.

An example of a manipulation interface that uses this indexed motion is the Cubic-Mouse, illustrated in Figure 2.21 [Fröhlich et al., 2000]. It consists of a plastic cubic structure of dimensions $9\text{ cm} \times 9\text{ cm} \times 9\text{ cm}$ ($3\frac{5}{8}\text{ in.} \times 3\frac{5}{8}\text{ in.} \times 3\frac{5}{8}\text{ in.}$). This plastic cube houses a Polhemus Fastrack, three mutually perpendicular translating rods, and six application-programmable control buttons. The buttons are placed on the face that normally is oriented upward (the "up" face), while wires from the tracker, the sensors embedded in the translating rods, and the buttons exit through the opposite ("down") face. (down) face.

As opposed to wands that are designed for flyby navigation, the Cubic-Mouse is designed primarily for the manipulation of a single, large virtual model. Navigation in this context means rotating and translating the virtual model through the tracker and dissecting/editing it using the three rods. Each of the rods controls a cutting plane, and buttons at the end of the rods allow for selecting of the orientation of the cutting planes. Further functionality is brought by the control buttons, one being dedicated to indexed motion (the clutch button) and two to zooming in-out on the virtual model. As opposed to classical computer mice, the Cubic-Mouse is designed for dual-hand operation. Users hold the mouse with their nondominant hand and push the rods and buttons with their dominant one. Proprioception (the body's own sense of position/motion) and tactile cues allow operation without looking at the rods.

Cubic-Mouse — Easy Explanation in Points

The page is explaining a 3D input device called the Cubic-Mouse, which is used for navigation and manipulation in Virtual Reality (VR).

1. What is the Cubic-Mouse?

- The Cubic-Mouse is a special 3D input/manipulation device.
- It is designed mainly for manipulating a large virtual model.
- Unlike a normal mouse, it allows the user to control objects in 3D space.

2. Structure of the Cubic-Mouse

The device consists of a small plastic cube approximately:

- 9 cm × 9 cm × 9 cm

Inside/around the cube are:

- A Polhemus Fastrak magnetic tracker
- Three mutually perpendicular translating rods
- Application-programmable buttons
- Clutch button
- Zoom buttons

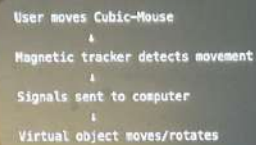
The diagram shows:



3. Magnetic Tracker

- The Cubic-Mouse contains a magnetic tracker.
- It tracks the position and orientation/movement of the device.
- This allows the computer to know how the user is moving the Cubic-Mouse.
- The tracker sends analog signals to the computer.

So:



4. Problem with Absolute Position Control

The text points out a problem with controlling virtual objects using absolute position.

- The user's physical arm has a limited reach.
- But the virtual object/display may be much larger.
- Therefore, the user cannot physically reach every required position.

For example:



The user's hand cannot physically cover the entire virtual model.

5. Solution — Motion Scaling

One solution is to use motion scaling.

- The movement of the user's hand is multiplied by a constant.
- A small physical movement can therefore produce a larger virtual movement.

For example:

Physical movement = 2 cm

× 5 scaling

Virtual movement = 10 cm

So:

Small hand movement → Large virtual-object movement

But there is a drawback:

- The user's tracker noise is also amplified.
- Small unwanted movements can therefore become larger.
- Selected objects may become jittery.

6. Alternative — Indexed Motion

Another solution is indexed motion.

The idea is:

Select an object → move it → stop → reposition your hand/device → continue moving it.

For example:

Select object

Move object

Reach limit of arm

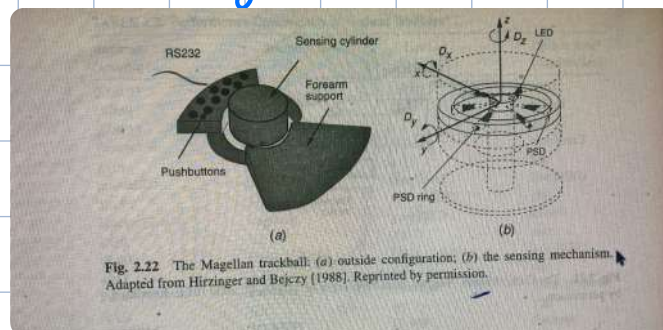
Stop object

Move hand back

Ask anything

→ Trackballs:

- A class of interfaces that allow navigation/manipulation in relative coordinates are trackballs
- this is a sensorized cylinder that measures 3 forces and 3 torques
- compliant



- central part of cylinder - fixed
- 6 LEDs placed on the ext moving cylinder
- when the user applies force or torque on the moving shell, the photo sensor output is used to measure 3 forces and 3 torques.

↓
sent to computer via RS 232 serial line.

- several buttons are integrated with the trackball support, within reach of the user's fingers.

↓
binary on/off

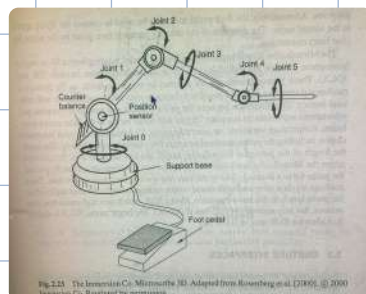
- trackballs suffer from sensor couplings.

↳ if user may want only translation but not rotation, the VR object may do both translation and rotation.

↓
due to non-zero sensed torques

↓
SOLUTION: software or hardware filters

→ 3D probes:



2.2.3 Three-Dimensional Probes

Users felt a need for an I/O device that would be intuitive to use, inexpensive, and allow either absolute or relative position control of the simulation. One such device is the Immersion Probe produced by Immersion Co. in the early 1990s. This was later renamed the MicroScribe 3D and its use extended to include digitizing objects [Rosenberg et al., 2000]. It consists of a small, sensorized mechanical arm that sits on a support base, with a small 6 in. $\times$ 6 in. footprint. The probe has six joints (joints 0–5), as illustrated in Figure 2.23.

Each rotary joint represents one degree of freedom, and thus the probe has six degrees of freedom, allowing simultaneous positioning and orienting of its tip. A counterbalance is placed close to the base to minimize the user's fatigue. The tip position relative to the base is obtained through direct kinematics calculations, based on sensor values and the length of the links. Software on the host computer reads the joint sensors on an RS232 line, then uses its kinematic model to determine where the tip is. A binary switch on a foot pedal is used to select/deselect (release) virtual objects, navigate (start/stop), or mark a point on the real object surface for digitization

purposes. Alternatively, the foot pedal may also be used to control the flying speed in the virtual scene. The direction of the velocity vector is then given by the probe tip (last link) orientation.

The MicroScribe 3D workspace depends on its model, ranging from a 50-in.-radius hemisphere (models 3D and 3DX) to a 66-in.-radius hemisphere (models 3DL and 3DLX). These measurements are not affected by magnetic fields, as the probe is a mechanical structure. It is thus possible to measure over 1000 positions/sec with minimal latency. The difference between the probe's actual and measured positions and orientations represents its accuracy. Errors will occur owing to the joint sensor resolution, temperature variations, or joint tolerances. Unfortunately, all open kinematic chains, including the probe, accumulate errors from the base toward the tip. The larger the length of the particular MicroScribe model, the less accurate it is. To minimize errors the MicroScribe needs calibration before use. Calibration is done by placing the probe tip in a housing close to the base and reading the rotary sensor data. These readings are then compared with stored values corresponding to the joint angles when the probe arm is in this known position. Model 3DX, which uses high-resolution joint sensors, has a tip accuracy of 0.23 mm (0.009 in.). The larger model 3DLX is slightly less accurate (0.30 mm, or 0.012 in.).

GESTURE INTERFACES:

[Definition]: Gesture Interfaces are devices that measure the real-time position of the user's fingers in order to allow natural, gesture-recognition-based interaction with the virtual environment.

2.3 GESTURE INTERFACES

⑥

Trackballs and 3D probes have the advantage of simplicity, compactness, and quiet operation. By their nature they limit the freedom of motion of the user's hand to a small area close to the desk. Therefore the natural motion of the user's hand is sacrificed, and interaction with the virtual world is less intuitive. In order to have large gesture-based interactions with the VR simulation, it is necessary that the I/O devices maintain the hand freedom of motion within a certain volume. It is also desirable to allow additional degrees of freedom by sensing individual finger motions. The human fingers have degrees of freedom associated with flexion-extension and lateral abduction-adduction, as illustrated in Figure 2.24 [American Society for Surgery of the Hand, 1983]. Additionally, the thumb has an anteposition-retroposition motion, which brings it in opposition to the palm.

Definition Gesture interfaces are devices that measure the real-time position of the user's fingers (and sometimes wrist) in order to allow natural, gesture-recognition-based interaction with the virtual environment.

Most gesture interfaces today are sensing gloves that have embedded sensors which measure the position of each finger versus the palm. Sensing gloves differ in such factors as, for example, the type of sensors they use, the number of sensors for each finger (one or several), their sensor resolution, the glove sampling rate, and whether they are tethered or wireless.

Some of the available commercial sensing gloves are the Fakespace Pinch Glove [McDowall et al., 2000], the Fifth Dimension Technology 5DT Data Glove [Fifth Dimension, 2000], the Didjiglove [Anon, 2000], and the Immersion CyberGlove [Immersion Co, 2001]. They have sensors that measure some (or all) of the finger joint angles. Some have built-in trackers as well, in order to measure the user's

2.3 GESTURE INTERFACES 47

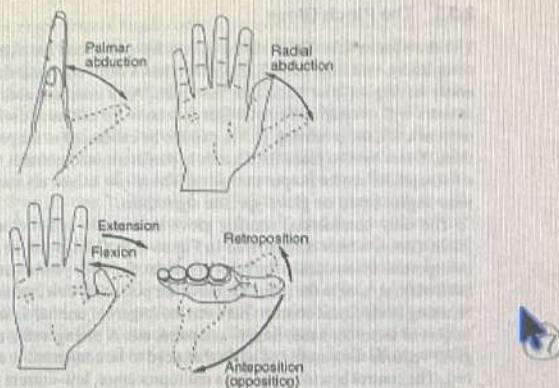


Fig. 2.24 Terminology of hand and finger motions. Adapted from American Society for Surgery of the Hand [1983]. ©1983 Elsevier Science. Reprinted by permission.

wrist motion. The resulting sensing glove work envelope is much larger than that of trackballs or joysticks, as illustrated in Figure 2.25. As opposed to trackballs and 3D probes, which have single-point interaction with the virtual environment, sensing gloves allow dextrous, multipoint interaction at the fingertips or palm. This results in a more realistic simulation, especially for object manipulation tasks. Additionally, sensing gloves can become navigation interfaces, based on user-programmed gesture libraries [Laviola, 2000].

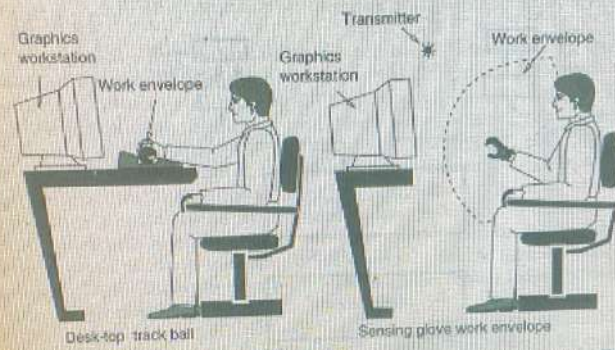
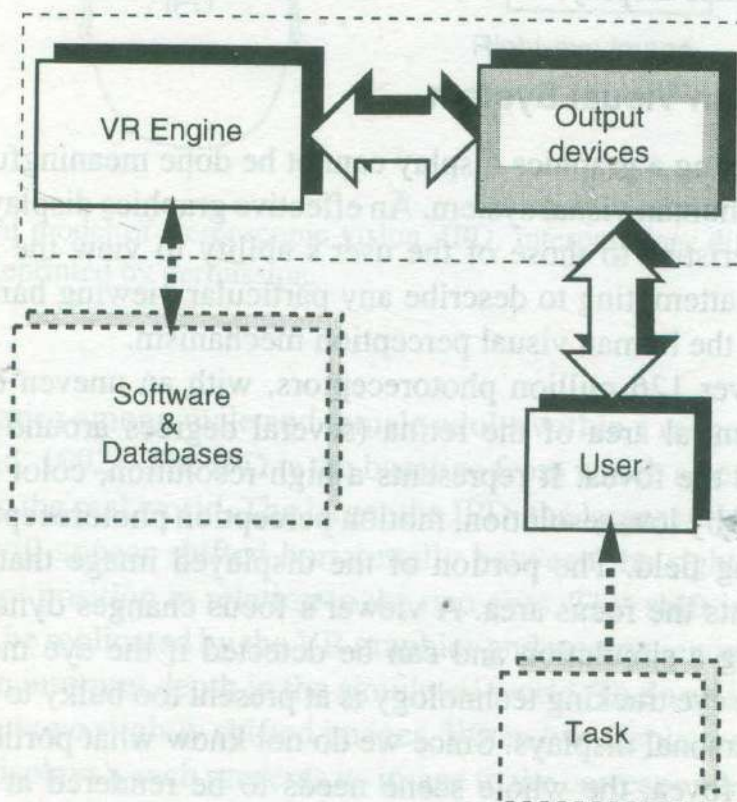


Fig. 2.25 Comparison of sensing glove work envelope and trackball work envelope. From Burdea [1993]. Reprinted by permission.

OUTPUT DEVICES: GRAPHICS, THREE-DIMENSIONAL SOUND, AND HAPTIC DISPLAYS

VR System Architecture



The previous chapter described 3D trackers, trackballs, and sensing gloves, which are devices used to mediate the user's input into the VR simulation. Now we look at special hardware designed to provide feedback from the simulation in response to this input. The sensorial channels fed back by these interfaces are sight (through graphics displays), sound (through 3D sound displays), and touch (through haptic displays).

For reasons of simplicity, this chapter treats each sensorial feedback modality separately. A further simplification here is the decoupling of sensorial feedback from the user's input. Modern VR systems are, however, multimodal, and feedback interfaces usually incorporate hardware to allow user input (such as trackers, pushbuttons, etc.). Combining several types of sensorial feedback in a simulation increases the user's immersion into VR. Furthermore, combining several sensorial modalities improves the simulation realism and therefore the usefulness of VR applications.

3.1 GRAPHICS DISPLAYS

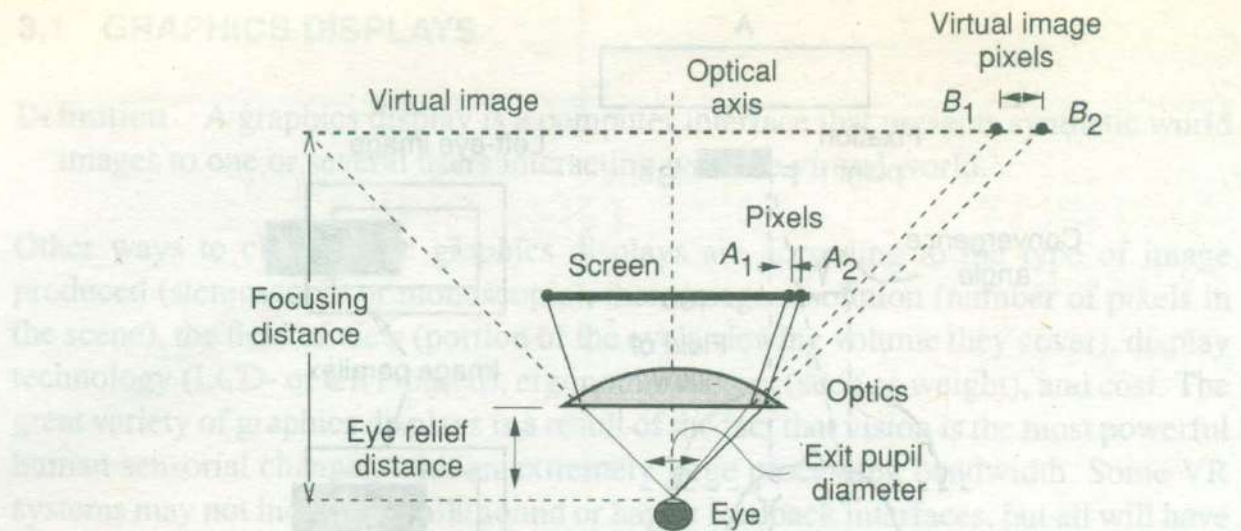


Fig. 3.2 Simplified optics model of an HMD. Adapted from Robinett and Rölland [1992]. ©1992 Massachusetts Institute of Technology. Reprinted by permission.

3.1.2 Personal Graphics Displays

Definition A graphics display that outputs a virtual scene destined to be viewed by a single user is called a personal graphics display. Such images may be monoscopic or stereoscopic, monocular (for a single eye), or binocular (displayed on both eyes).

Within the personal display category are grouped head-mounted displays (HMDs), hand-supported displays, floor-supported displays, and autostereoscopic monitors.

3.1.2.1 Head-Mounted Displays. These project an image floating some 1–5 m (3–15 ft) in front of the user [Anon, 2001], as illustrated in Figure 3.2. They use special optics placed between the HMD small image panels and the user's eyes in order to allow the eyes to focus at such short distances without tiring easily. Optics is also needed to magnify the small panel image to fill as much as possible of the eyes' field of view (FOV). As can be seen in Figure 3.2 [Robinett and Rolland, 1992], the unfortunate byproduct is that the distance between display pixels (A_1 , A_2) is amplified as well. Therefore the "granularity" of the HMD displays (expressed in arc-minutes/pixel) becomes apparent in the virtual image. The lower the HMD resolution and the higher the FOV, the greater is the number of arc-minutes of eye view corresponding to each pixel. Older HMDs had very low resolution (as discussed in Chapter 1) and incorporated a diffuser sheet overlaid on the input to their optics. These diffusers blurred the edges between pixels somewhat, and the image was perceived to be better. Modern HMDs have resolutions up to extended video graphics array (XVGA; 1024×768) or better. Even these resolutions may be unsatisfactory if the optics amplifies the pixel dimension too much in order to enlarge the apparent FOV. Furthermore, very large FOVs force large exit pupil diameters, which can produce shadows around the edges of the perceived image.

Another HMD characteristic is the display technology used. Consumer-grade HMDs use LCD displays, while more expensive professional-grade devices incorporate CRT-based displays, which tend to have higher resolution. Since consumer HMDs are designed primarily for private viewing of TV programs and for video games rather

than for VR, they accept NTSC (in Europe, phase alternating line, PAL) monoscopic video input. When integrated within a VR system, they require the conversion of the graphics pipeline output signal red–green–blue (RGB) format to NTSC/PAL, as illustrated in Figure 3.3a. The HMD controller allows manual adjustment for brightness and forwards the same signal to both HMD displays. Professional-grade HMDs are designed specifically for VR interaction and accept RGB video input. As illustrated in Figure 3.3b, in the graphics pipeline two RGB signals are sent directly to the HMD control unit (for stereoscopic viewing). The control unit also receives stereo sound, which is then sent to the HMD built-in headphones. The user's head motion is tracked and position data sent back to the VR engine for use in the graphics computations. A ratchet on the back of the HMD head support allows comfort adjustment for various head sizes.

The HMD weight, comfort, and cost are additional criteria to be considered in comparing the various models on the market. Technology has made tremendous advances in miniaturizing the HMD displays and associated optics. The first LCD-based HMD (the VPL EyePhone), which was available in the early 1990s, had a resolution of 360×240 pixels, a FOV of 100° horizontally and 60° vertically, and a weight of 2.4 kg (2400 grams). This large weight induced user fatigue. Modern LCD-based

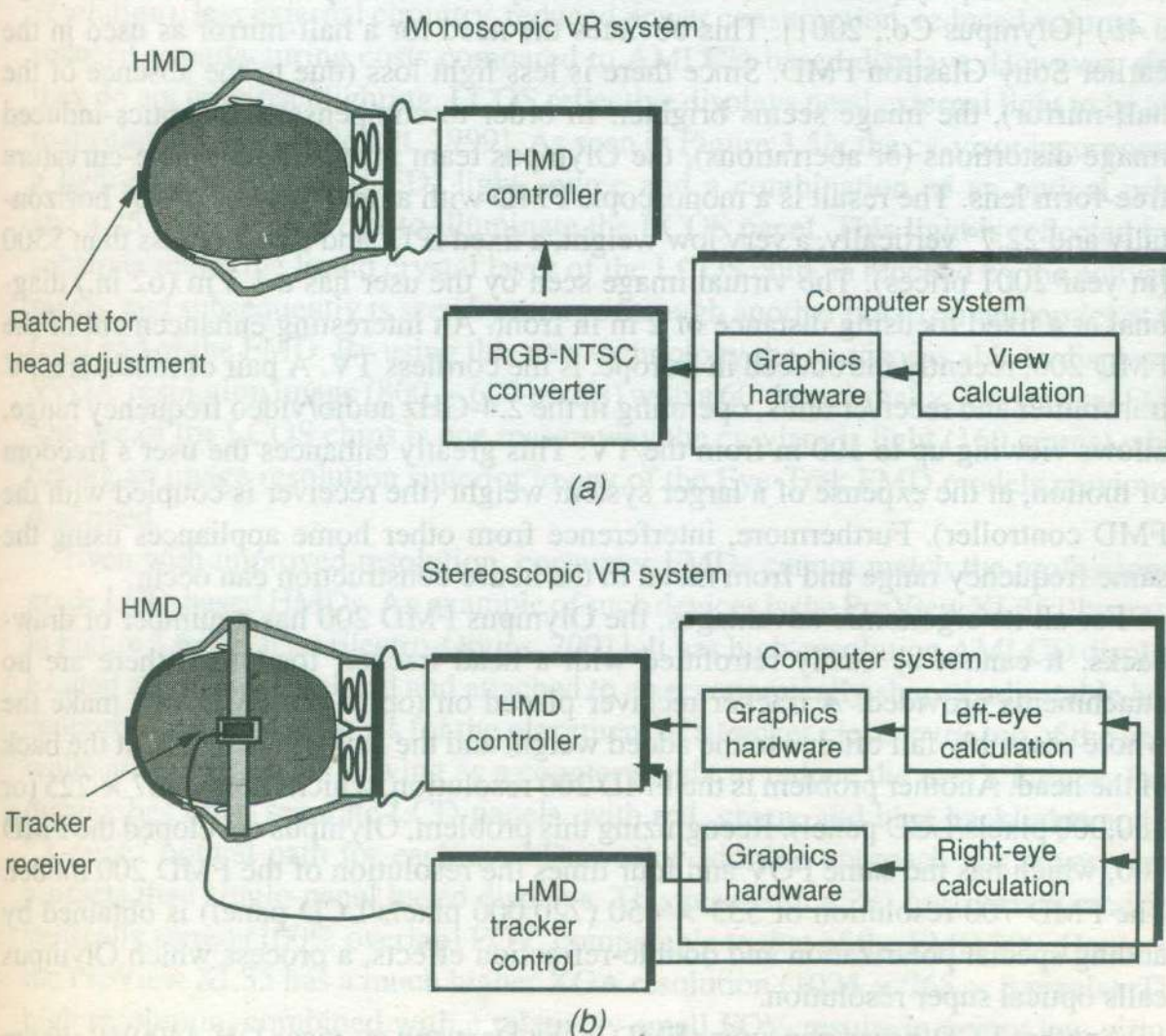


Fig. 3.3 Head-mounted display (HMD) integration in a VR system for: (a) consumer (monoscopic) HMD; (b) professional (stereoscopic) HMD. Adapted in part from Pimentel and Teixeira [1993]. Reprinted by permission.

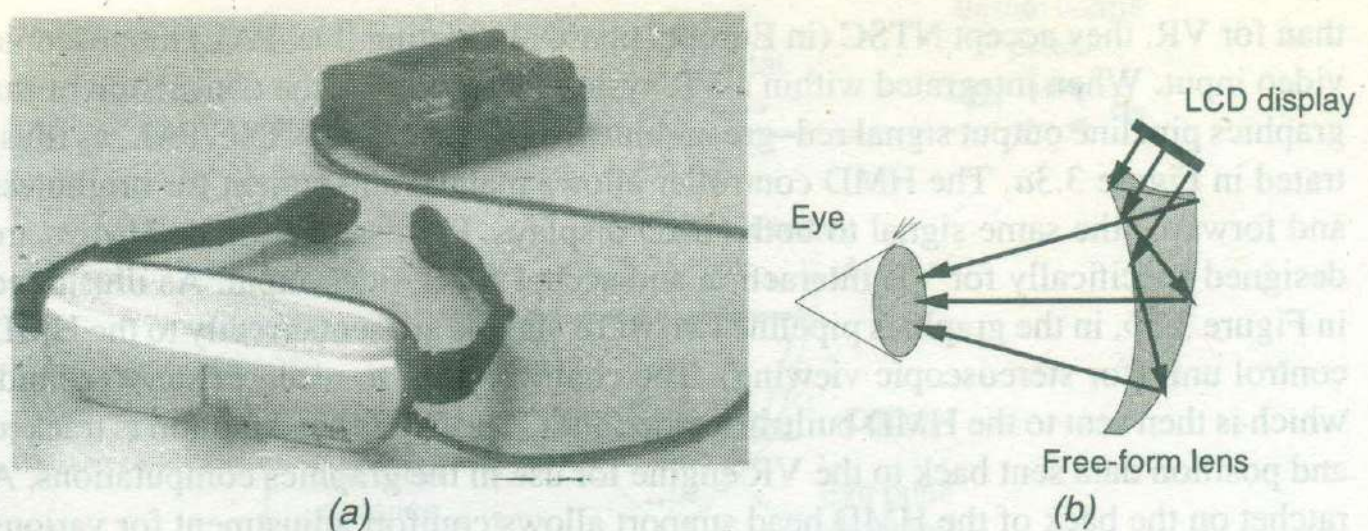


Fig. 3.4 The Olympus LCD-based Eye-Trek FMD: (a) the FMD 200; (b) optics arrangement. Adapted from Olympus Co. [2001]. Reprinted by permission.

HMDs, such as the Olympus Eye-Trek, shown in Figure 3.4a [Isdale, 2000], weighs only about 100 grams (24 times less!). This class of HMDs resembles eyeglasses, and so they are also called face-mounted displays (FMDs). Key to the compactness and lightness of the Olympus FMD is the placement of its small active matrix LCD (AMLCD) display panels eccentrically, directly above the optics (as shown in Figure 3.4b) [Olympus Co., 2001]. This obviates the need for a half-mirror as used in the earlier Sony Glastron FMD. Since there is less light loss (due to the absence of the half-mirror), the image seems brighter. In order to compensate for optics-induced image distortions (or aberrations), the Olympus team invented a variable-curvature free-form lens. The result is a monoscopic FMD with a field of view of 30° horizontally and 22.7° vertically, a very low weight, a fixed IPD, and a cost of less than \$500 (in year 2001 prices). The virtual image seen by the user has a 1.3 m (62 in.) diagonal at a fixed focusing distance of 2 m in front. An interesting enhancement of the FMD 200, recently introduced in Europe, is the cordless TV. A pair of miniature TV transmitter and receiver units, operating in the 2.4-GHz audio/video frequency range, allows viewing up to 100 m from the TV. This greatly enhances the user's freedom of motion, at the expense of a larger system weight (the receiver is coupled with the FMD controller). Furthermore, interference from other home appliances using the same frequency range and from metal in the house construction can occur.

For all its ergonomic advantages, the Olympus FMD 200 has a number of drawbacks. It cannot be easily retrofitted with a head tracker, for which there are no attachments provided. A tracker receiver placed on top of the FMD will make the whole assembly fall off due to the added weight and the lack of a restraint at the back of the head. Another problem is the FMD 200 resolution, which is only 267×225 (or 180,000 pixels/LCD panel). Recognizing this problem, Olympus developed the FMD 700, which has the same FOV and four times the resolution of the FMD 200 model. The FMD 700 resolution of 533×450 (720,000 pixels/LCD panel) is obtained by adding special polarization and double-refraction effects, a process which Olympus calls optical super resolution.

Another LCD-based consumer FMD is the Daeyang cy-visor DH-4400VP shown in Figure 3.5a. Although it looks somewhat similar to the Eye-Trek FMD, the cy-visor uses liquid-crystal-on-silicon (LCOS) display technology. Classic AMLCD panels are transmissive, meaning that light passes through the electrode/liquid crystal/electrode

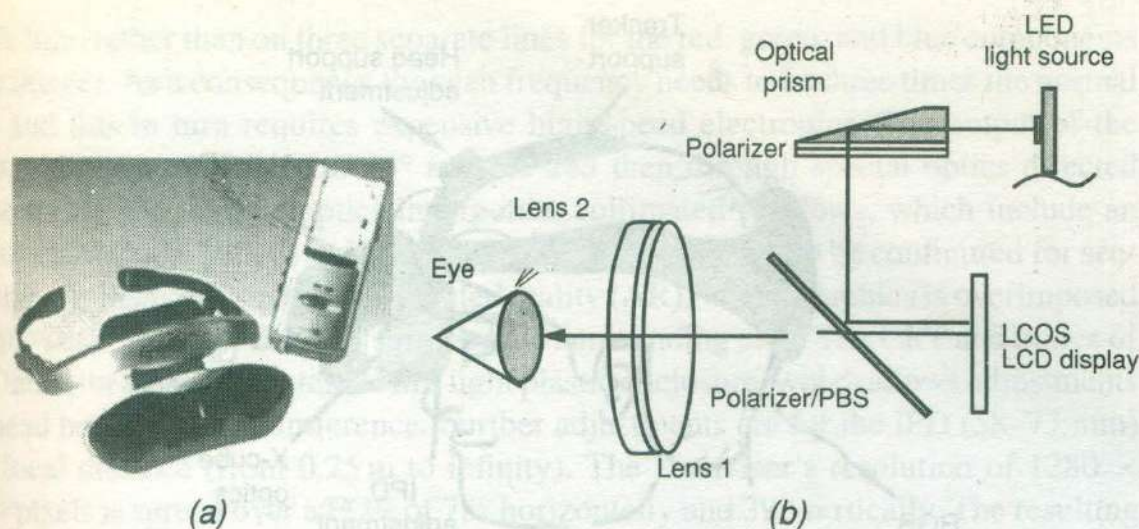


Fig. 3.5 The Daeyang cy-visor FMD: (a) the DH-4400VP; (b) optics arrangement. Adapted from Daeyang [2001]. Reprinted by permission.

substrate and is then seen by the user. The great advantage of newer LCOS LCD displays is that the liquid crystal layer is mounted directly on a silicon chip, which is used as control circuitry. This results in higher electrode densities (i.e., display resolution), less external circuitry, reduced power consumption, reduced volume, and reduced manufacturing costs compared to AMLCD-based displays. However, since they do not have backlighting, LCOS reflective displays need external light to be able to display the image [Schott, 1999]. As seen in Figure 3.5b, the cy-visor incorporates a light-emitting diode (LED) light source and a combination of an optical prism and a 45° mirror/polarizer to illuminate the LCOS panel. This light is reflected by a substrate under the liquid crystal layer of the LCOS chip, is blocked by the activated pixels, and subsequently is seen by the eye through another polarizer and optics at the front end of the FMD. By using this new technology the cy-visor is able to display an SVGA-resolution image (800×600 pixels) with a 60° horizontal $\times$ 43° vertical FOV. The use of the LCOS chips is one reason why the cy-visor is light (160 grams) while having an image resolution superior to any of the Eye-Trek FMD models previously described.

Even with improved resolution, consumer FMDs cannot match the professional-grade LCD-based HMDs. An example of such devices is the ProView XL35 illustrated in Figure 3.6a [Kaiser Electro-Optics, 2001]. It has high-resolution AMLCD displays located at the user forehead and attached to an ergonomically shaped adjustable head support. The support allows for the placement of a tracker close to the top of the head, while at the same time serving as a counterweight to reduce the user's fatigue. Each display has three separate LCD panels, with red, green, and blue backlights and an "X-cube" optical path for each eye. This color addition approach has fewer visual artifacts than single-panel based displays. This results in a 28° horizontally and 21° vertically stereo (100% overlap) FOV, comparable to that of the FMD 200. However, the ProView XL35 has a much higher XGA resolution ($1024 \times 768 \times 3$ pixels). The high resolution, combined with a relatively small FOV, results in a very low virtual image granularity (1.6 arc-minutes/color pixel). Further advantages are the adjustable IPD (55–75 mm) and an eye relief that allows eyeglasses to be worn together with the ProView HMD. The high image quality of the ProView XL35 comes at a price in

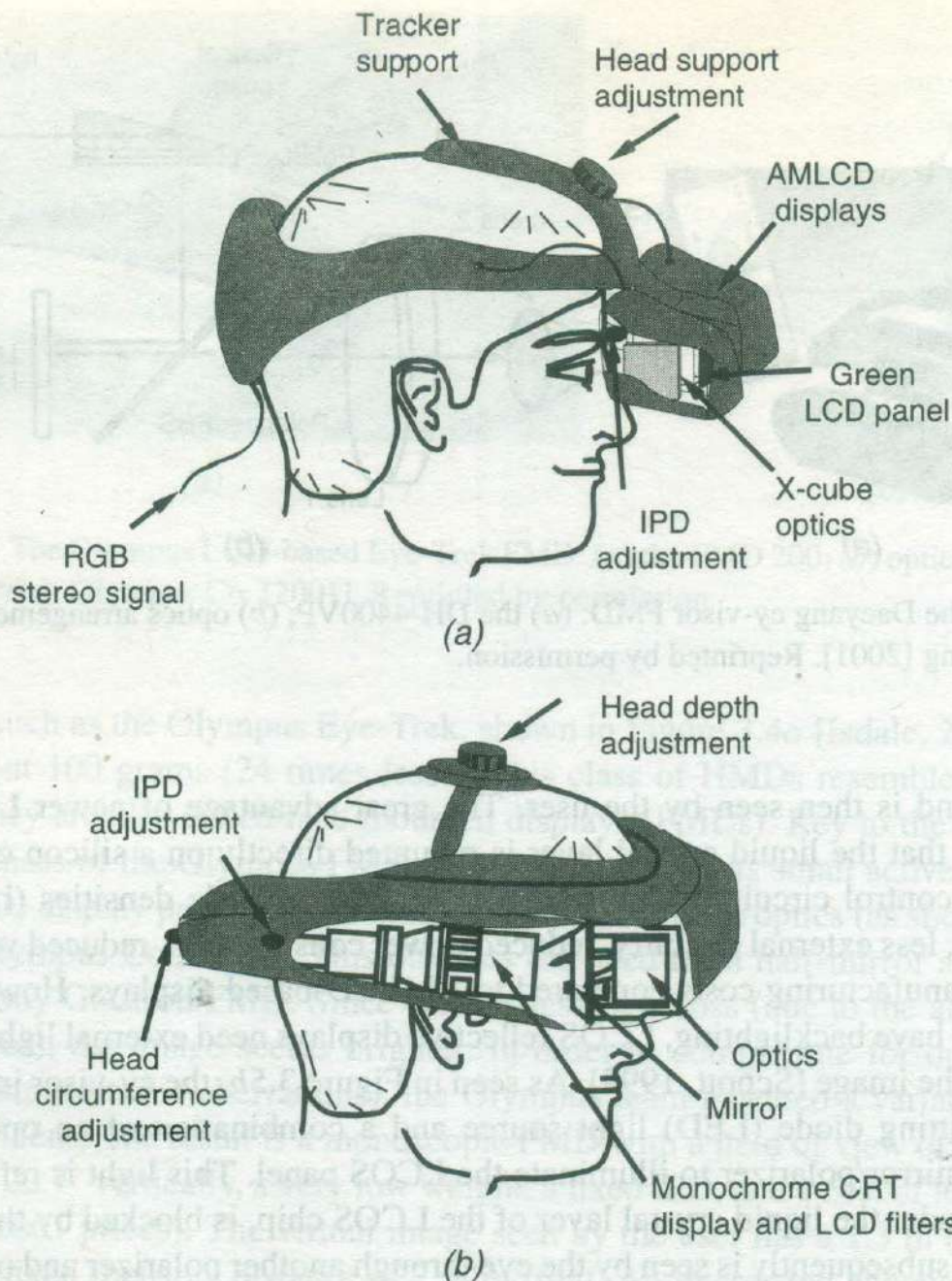


Fig. 3.6 Stereoscopic HMDs. (a) ProView XL35. Adapted from Kaiser Electro-Optics [2001]. Reprinted by permission. (b) Datavisor HiRes. Adapted from n-vision Inc. [1998].

terms of much higher cost (\$19,500) and weight (992 grams, or 35 oz) compared to those of the FMD 200.

The high resolution of the ProView XL35 HMD represents only 60% of that available on desk-top graphics monitors, which typically have 1280×1024 color pixels. To obtain such display resolutions, today's high-end HMDs use miniature (1-in. diameter) CRTs. An example is the Datavisor HiRes, illustrated in Figure 3.6b [n-vision Inc., 1998]. Its two monochrome (black/white) CRTs are placed laterally to the user's head and retrofitted with small Tektronix Nucolor liquid crystal shutters. The CRTs and shutter units are metal-shielded to eliminate electronic interference and to dissipate heat away from the user's head. Special care is required in the electrical circuitry because of the high voltage of the CRTs, which are close to the user's head.

By rapidly switching the filters, a single line in the Datavisor's image is first seen in red, then in green, and finally in blue. The human brain integrates the three images to obtain a color scene. This approach in turn requires the so-called field-sequential video input format, meaning that RGB color information is transmitted sequentially on a

single line (rather than on three separate lines for the red, green, and blue components of an image). As a consequence the scan frequency needs to be three times the normal rate, and this in turn requires expensive high-speed electronics. The output of the LCD shutters is reflected off 45° mirrors and then through special optics directed to the user's eyes. These optics incorporate collimated windows, which include an undistorted optical path to the eye. This allows the Datavisor to be configured for see-through applications, such as augmented reality (AR), where graphics is superimposed on the visual image of the user's immediate surrounding area. The CRT and optics of the Datavisor are fully contained in a light plastic enclosure, which allows adjustments for head height and circumference. Further adjustments are for the IPD (58–73 mm) and focal distance (from 0.25 m to infinity). The Datavisor's resolution of 1280×1024 pixels is spread over a FOV of 78° horizontally and 39° vertically. The resulting image granularity is 1.9 arc-minutes/color pixel, which is about 20% worse than that of the ProView XL35 LCD HMD. Other drawbacks of the Datavisor are large weight (1587 grams, or 56 oz) and unit cost (\$35,000). This is further complicated by safety concerns related to the CRT electromagnetic radiation close to the brain.

the resolution drops by orders of magnitude. Increasing the resolution of very large displays to that of workstations would involve many more projectors and an even larger cost than those given in Table 3.2.

3.2 SOUND DISPLAYS

Definition Sound displays are computer interfaces that provide synthetic sound feedback to users interacting with the virtual world. The sound can be monoaural (both ears hear the same sound) or binaural (each ear hears a different sound).

Sound displays play an important role in increasing the simulation realism by complementing the visual feedback provided by the graphics displays previously discussed. Assume a user is looking at a virtual ball bouncing in a virtual room that is displayed on a CRT monitor. The user's mind says that he or she should also hear a familiar "plop-plop-plop" sound. When sound is added, the user's interactivity, immersion, and perceived image quality increase. In this scenario simple monoaural sound is sufficient, as the ball is always in front of the user and being displayed by the monitor. Let us now consider another user, who looks at the same virtual world, this time displayed by an HMD. If the ball bounces away, outside the field of view (as balls in the real world sometimes do), then the user cannot tell where the ball went based on visual information or on monoaural sound alone. Now the simulation needs a device that provides binaural sound in order to localize the "plop-plop-plop" sound in 3D space relative to the user.

This example illustrates an important distinction within the sound feedback modality. Highly immersive virtual reality simulations should have 3D sound, also called virtual sound, in addition to graphics feedback. The distinction between 3D sound and stereo sound is illustrated in Figure 3.20 [Burdea and Coiffet, 1993]. Stereo sound on headphones seems to emanate inside the user's head. In other words it is not externalized, as real sounds are. When the user wears simple stereo headphones, the violin will turn to the left following the user's head motion. A 3D sound presented on the same headphones or on speakers contains significant psychoacoustic information to alter the user's perception into believing that the recorded sound is actually coming from the user's surroundings [Currell, 1992]. In Figure 3.20, the 3D sound is synthesized using head tracker data, and the virtual violin remains localized in space. Therefore its sound will appear to move to the back of the user's head. Sound in a real room bounces off the walls, the floor, and the ceiling, adding to the direct sound received from the violin. The realism of the virtual room therefore requires that these reflected sounds be factored in.

3.2.1 The Human Auditory System

Three-dimensional sound displays cannot be effective without an understanding of the way we localize sound sources in space. Humans perceive sound through vibrations arriving at the brain via the skeletal system or via the ear canal. Within the scope of this book we are interested in the ear's ability to detect the position of a sound source relative to the head.

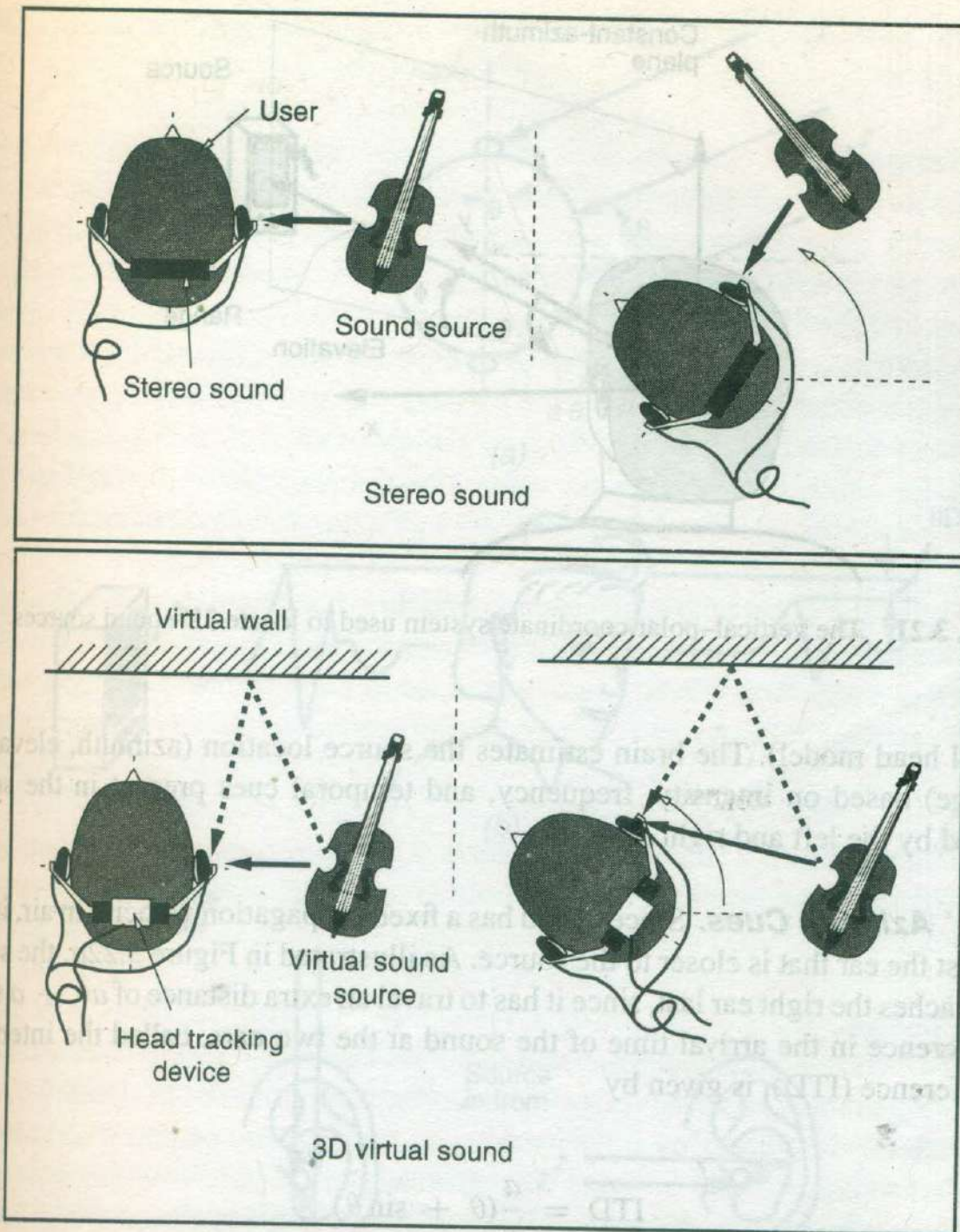


Fig. 3.20 Stereo sound versus 3D virtual sound. From Burdea and Coiffet [1993]. © Editions Hermes. Reprinted by permission.

3.2.1.1 The Vertical-Polar Coordinate System. In order to measure position it is necessary to establish a head-attached system of coordinates. If such a system of coordinates is Cartesian, then the location of a sound source relative to the head can be expressed by the triad (x, y, z) . Alternatively, the same 3D sound source position can be expressed in a spherical system of coordinates called the vertical-polar coordinate system [Duda, 1997]. As illustrated in Figure 3.21, the sound source location is uniquely determined by three variables, namely azimuth, elevation, and range. The azimuth is the angle θ between the nose and a plane containing the source and the vertical axis z . The source elevation is the angle ϕ made with the horizontal plane by a line passing through the source and the center of the head. Finally, the range r is the distance to the source measured along this line. The sound source azimuth can vary anywhere between $\pm 180^\circ$, while the elevation has a range of $\pm 90^\circ$. Finally, source range can only be positive and larger than the head radius (assuming a simplified

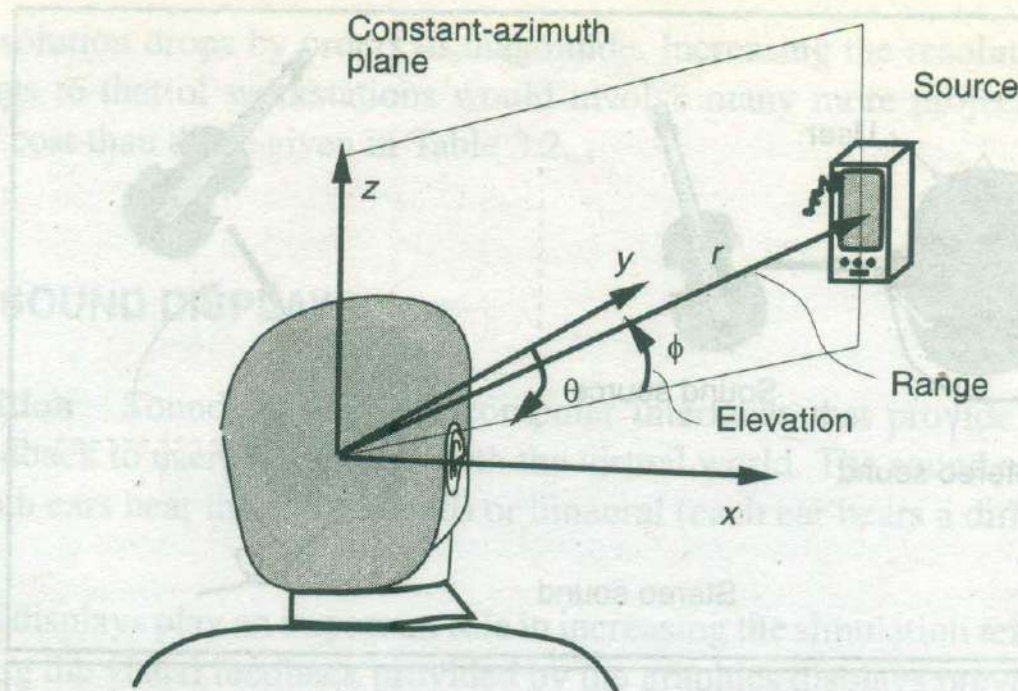


Fig. 3.21 The vertical-polar coordinate system used to locate 3D sound sources.

spherical head model). The brain estimates the source location (azimuth, elevation, and range) based on intensity, frequency, and temporal cues present in the sound perceived by the left and right ears.

3.3 HAPTIC FEEDBACK

(X)

The last category of I/O devices discussed in this chapter are haptic interfaces. Named after the Greek term *hapthai* (meaning “touch”), they convey important sensorial information that helps users achieve tactile identification of virtual objects in the environment and move these objects to perform a task [Cutt, 1993; Burdea, 1996]. When added to the visual and 3D audio feedback previously discussed, haptic feedback greatly improves simulation realism. It becomes mandatory wherever the visual feedback is corrupted (occluded objects that need to be manipulated) or lacking entirely (dark environments). The discussion in this chapter is limited to feedback to

the user's hand and wrist because the hand has the highest density of touch receptors in the body. Furthermore, systems that provide feedback to the whole body are either in the research stage at this time or are too specialized to be of use outside their intended settings.

Haptic feedback groups the two modalities of touch and force feedback. Consider a hand pushing very lightly against a desk. The sensors that respond first are the fingertip touch sensors. If the hand pushes harder, then the muscles in the hand and forearm start to contract. The level of force applied is now felt by sensors on muscle ligaments and bones (kinesthesia) as opposed to fingertips. The technical literature intermixes the terms "touch" and "force" feedback, so clarification is needed.

Definition Touch feedback conveys real-time information on contact surface geometry, virtual object surface roughness, slippage, and temperature. It does not actively resist the user's contact motion and cannot stop the user from moving through virtual surfaces.

Definition Force feedback provides real-time information on virtual object surface compliance, object weight, and inertia. It actively resists the user's contact motion and can stop it (for large feedback forces).

Designing good haptic feedback interfaces is a daunting task in view of the specific requirements they need to meet. The first is user safety and comfort. While the user interacts with virtual objects, the forces he or she feels are *real*. These contact forces need to be large (in the simulation of rigid objects), but not large enough to harm the user. In this context a good design is also fail-safe, so that users are not subject to accidents in case of computer failure. Yet another issue is portability and user comfort. The difficulty with force-feedback actuators is the need to provide sufficient force while still keeping the feedback hardware light and unintrusive. If haptic interfaces are too heavy and bulky, then the user will get tired easily and will prefer a less cumbersome open-loop control. Heavy feedback structures can be gravity counterbalanced, but this further increases complexity and cost. Portability also relates to ease of use and installation at the simulation site. Haptic feedback interfaces should be self-contained, without requiring special supporting construction, piping, or wiring.

All these conflicting requirements call for a compromise, which can only be found if the human haptic characteristics are taken into account. In the context of this book we look at the sensorial characteristics of the hand from the engineering perspective. The hardware designer is interested in the manual force exertion capability, which helps in selecting the appropriate feedback actuators. The hardware designer is also interested in manual degrees of freedom and mobility, which determine the number of feedback actuators and their range of motion. Similarly, the control engineer is interested in the sensorial bandwidth of the hand, which determines how fast the control loop needs to be closed. Finally, the human factors specialist is interested in the safety and comfort issues related to feedback to the hand.

3.3.1 The Human Haptic System

Input to the human haptic system is provided by the sensing loop, while output to the environment (in this case the haptic interface) is mediated by the sensory-motor

control loop. The input data are gathered by a multitude of tactile, proprioceptive, and thermal sensors, while the output is forces and torques resulting from muscular exertion. The system is not balanced, in the sense that humans perceive haptically much faster than they can respond.

3.3.1.1 Haptic Sensing. The skin houses four types of tactile sensors (or receptors), namely Meissner corpuscles (the majority), Merkel disks, Pacinian corpuscles, and Ruffini corpuscles. When excited they produce small electrical discharges, which are eventually sensed by the brain. The slow-adapting (SA) sensors (Merkel and Ruffini) maintain a constant rate of discharge for a long time in response to a constant force being applied on the skin. On the contrary, the fast-adapting (FA) ones (Meissner and Pacinian) drop their rate of discharge so fast that the constant contact force becomes undetected. Such different behavior is related to the particular type of contact the tactile sensors detect. If the contact has a strong time variation (vibrations on the skin, or accelerations), then FA receptors are needed. Conversely, if the contact is quasi steady state (edges, skin stretch, static force), then SA receptors are used. As a consequence, Merkel and Ruffini receptors are most sensitive to low-frequency stimuli (0–10 Hz), while Meissner and Pacinian sensors respond to stimuli of higher frequencies (50–300 Hz). The receptive field of a tactile receptor determines its spatial resolution. If the sensor has a large receptive field, its spatial resolution is low. Conversely, the Meissner and Merkel receptors, which have small receptive fields, have high spatial resolution. Table 3.3 summarizes the characteristics of the four tactile sensors (or receptors) present in the skin [Burdea, 1996].

The spatial resolution of the skin varies depending on the density of receptors it has. The fingertips, where such density is the highest, can discriminate two contacts that are at least 2.5 mm apart. However, the palm cannot discriminate two points that are less than 11 mm apart, and the user feels as if only one contact point exists [Shimoga, 1992]. Spatial resolution is complemented by temporal resolution when two contacts occur on the skin close in time. The skin mechanoreceptors have a successiveness threshold of only 5 msec, which is much smaller than the corresponding value for the eye (25 msec).

Temperature sensing is realized by specialized thermoreceptors and nociceptors. The thermoreceptors are located either in the epidermis (sensitive to cold) or dermis (sensitive to warmth). Their spatial resolution is less than that of mechanoreceptors.

TABLE 3.3. Comparison of Various Skin Mechanoreceptors^a

Receptor Type	Rate of Adaptation ^b	Stimulus Frequency (Hz)	Receptive Field	Detection Function
Merkel disks	SA-I	0–10	Small, well defined	Edges, intensity
Ruffini corpuscles	SA-II	0–10	Large, indistinct	Static force, skin stretch
Meissner corpuscles	FA-I	20–50	Small, well defined	Velocity, edges
Pacinian corpuscles	FA-II	100–300	Large, indistinct	Acceleration, vibration

^aBased on Seow [1988], Cholewiak and Collins [1991], and Kalawsky [1993].

^bSA, slow-adapting; FA, fast-adapting.